

Лабораторная работа 1. Начало работы с MySQL. MySQL Workbench

Задание 1

Раздел "Management" ("Управление")

1. Раздел "Server Status". В разделе отображается общая информация о сервере и подключении к нему. Информация логически сгруппирована. Можно выделить следующие группы:
 - a. Общая информация (например, название хоста, номер порта, версия БД).
 - b. Настройки сервера (например, включен ли брандмауэр, используется ли SSL)
 - c. Каталоги сервера
 - d. Сводка по используемым ресурсам компьютера (ОЗУ, процессор и т. д.)
 - e. Настройки соединения SSL (если SSL включена).

Раздел "Instance" ("Экземпляр БД")

1. Раздел "Startup / Shutdown" ("Запуск / Остановка").
 - a. Позволяет удаленно запустить или остановить службу MySQL Server. Для этого требуются права администратора на целевом сервере. Интерфейс отображает текущее состояние сервера (работает/остановлен) и предоставляет соответствующие кнопки для управления.
2. Раздел "Status and System Variables" ("Статус и системные переменные").
 - a. System Variables (Системные переменные): Отображает все настройки сервера (например, innodb_buffer_pool_size, max_connections). Переменные можно фильтровать и искать. Многие из них можно изменять непосредственно в интерфейсе ("динамические переменные"), что применяется сразу для текущей сессии или с возможностью сохранения в конфигурационный файл для применения после перезагрузки сервера.
 - b. Status Variables (Переменные состояния): Показывает текущую статистику работы сервера (например, количество активных подключений Threads_connected, число запросов Queries, объем отправленных и полученных данных). Эти переменные только для чтения и помогают оценить текущую нагрузку.
3. Раздел "Options File" ("Файл опций").
 - a. Предоставляет графический интерфейс для редактирования основного конфигурационного файла сервера (обычно my.cnf или my.ini). Это гораздо удобнее, чем ручное редактирование текстового файла. Настройки сгруппированы по категориям (General,

Networking, Security, Log Files и т.д.). Изменения, как правило, сохраняются в файл и вступают в силу после перезагрузки сервера.

4. Раздел "User Management" ("Управление пользователями").
 - a. Инструмент для администрирования учетных записей пользователей MySQL и их привилегий. Позволяет:
 - b. Создавать новых пользователей и удалять существующих.
 - c. Назначать пароли.
 - d. Управлять правами доступа на разных уровнях: глобальные привилегии, привилегии для конкретной базы данных, таблицы или даже столбца.
 - e. Настраивать ограничения на ресурсы (максимум запросов в час, максимум соединений в час и т.д.).
 - f. Назначать SSL-настройки для учетной записи.
5. Раздел "Server Logs" ("Журналы сервера").
 - a. Окно для просмотра в реальном времени различных журналов, которые ведет MySQL-сервер. Обычно доступны следующие типы логов:
 - Error Log (Журнал ошибок): Самый важный журнал, содержит информацию о запуске и остановке сервера, критические ошибки и предупреждения.
 - General Query Log (Общий журнал запросов): Логирует все запросы, поступающие к серверу. Используется для отладки, но может создавать большую нагрузку и обычно не включается на постоянной основе.
 - Slow Query Log (Журнал медленных запросов): Записывает запросы, выполнение которых превысило заданное пороговое значение времени. Ключевой инструмент для оптимизации производительности.
 - Binary Log (Бинарный журнал): Логирует все операции, изменяющие данные (DML и DDL). Используется для репликации и Point-in-Time-Recovery.

Раздел "Performance" ("Производительность")

1. Раздел "Performance Dashboard" ("Панель производительности").

Представляет собой сводную панель управления (dashboard) с ключевыми метриками производительности сервера в виде графиков и диаграмм. Сюда обычно входят:

 - a. Загрузка CPU сервера.
 - b. Использование оперативной памяти.
 - c. Статистика ввода/вывода (I/O).
 - d. Активность подключений.
 - e. Количество запросов в секунду.

- f. Информация из InnoDB Status (буферный пул, операции чтения/записи и пр.). Панель дает быстрое визуальное представление о "здоровье" сервера.
2. Раздел "Performance Reports" ("Отчеты о производительности").
- a. Предоставляет доступ к расширенным отчетам, которые генерируются на основе данных словаря производительности (Performance Schema) и системных переменных состояния. Эти отчеты помогают провести глубокий анализ. Примеры отчетов:
- Host Summary: Сводка по нагрузке, создаваемой с каждого хоста-источника подключений.
 - User Summary: Сводка по активности пользователей.
 - IO Summary: Детальный анализ операций ввода-вывода.
 - InnoDB Metrics: Подробные метрики, связанные с движком хранения InnoDB.
 - Query Analysis: Анализ выполнения запросов, включая отчеты из `sys`-схемы.
3. Раздел "Query Statistics" ("Статистика запросов").
- a. Этот инструмент фокусируется на анализе выполнения SQL-запросов. Он использует данные из `performance\_schema` для отображения статистики по запросам, сгруппированным по их шаблону (digest). Позволяет определить самые частые, самые медленные и самые ресурсоемкие запросы в системе, что является первым шагом к их оптимизации.
4. Раздел "Database Connections" ("Подключения к БД").
- a. Показывает список всех текущих подключений (поток) к серверу MySQL. Для каждого подключения можно увидеть:
- Идентификатор потока (Thread ID).
 - Пользователя и хост, с которого установлено подключение.
 - Базу данных по умолчанию.
 - Время выполнения текущего запроса и сам запрос (если он выполняется).
 - Состояние потока.
- b. Администратор может использовать этот инструмент для идентификации "зависших" или проблемных соединений и при необходимости принудительно завершить их (команда KILL).

Задание 3

```
CREATE TABLE `simplifiedb`.`users` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(45) NOT NULL,
```

```
`email` VARCHAR(45) NOT NULL,  
PRIMARY KEY (`id`),  
UNIQUE INDEX `userscol_UNIQUE` (`email` ASC) VISIBLE);
```

Задание 4

```
INSERT INTO `simplifiedb`.`users` (`name`, `email`) VALUES ('Lena',  
'Bai_len2006@mail.ru');  
INSERT INTO `simplifiedb`.`users` (`name`, `email`) VALUES ('Katya',  
'Huk_Kat1983@mail.ru');  
INSERT INTO `simplifiedb`.`users` (`name`, `email`) VALUES ('Dima',  
'Kij_Dima2003@mail.ru');
```

Задание 5

Поле `created` с типом `TIMESTAMP` и значением по умолчанию `CURRENT_TIMESTAMP()` автоматически фиксирует момент создания записи. `TIMESTAMP` хранит дату и время с учетом часового пояса, преобразуя их в UTC для хранения. При вставке новой записи, если значение для `created` не указано явно, в это поле автоматически записывается текущая дата и время с сервера БД. Это обеспечивает автоматическое и точное логирование времени добавления каждой записи в таблицу.

Поля, которые пользователь может захотеть не заполнять и они могут допускать значение `NULL`: `gender`, `bday`, `postal_code` и `rating`.

```
ALTER TABLE `simplifiedb`.`users`  
ADD COLUMN `gender` ENUM('M', 'F') NULL AFTER `email`,  
ADD COLUMN `bday` DATE NULL AFTER `gender`,  
ADD COLUMN `postal_code` VARCHAR(10) NULL AFTER `bday`,  
ADD COLUMN `rating` FLOAT NULL AFTER `postal_code`,  
ADD COLUMN `created` TIMESTAMP NOT NULL AFTER `rating`,  
CHANGE COLUMN `name` `name` VARCHAR(50) NOT NULL ;
```

Задание 7

```
-- Query: SELECT * FROM simplifiedb.users  
LIMIT 0, 1000
```

```
-- Date: 2025-09-21 00:09
```

```
*/
```

```
INSERT INTO ``  
(`id`, `name`, `email`, `gender`, `bday`, `postal_code`, `rating`, `created`) VALUES  
(1, 'Lena', 'Bai_len2006@mail.ru', NULL, NULL, NULL, NULL, '0000-00-00  
00:00:00');
```

```

INSERT INTO ``
('id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES
(2,'Katya','Huk_Kat1983@mail.ru',NULL,NULL,NULL,NULL,'0000-00-00
00:00:00');
INSERT INTO ``
('id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES
(3,'Dima','Kij_Dima2003@mail.ru',NULL,NULL,NULL,NULL,'0000-00-00
00:00:00');
INSERT INTO ``
('id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES
(4,'Alex','Alex_Gj2007@mail.ru','M',NULL,'575678',1.21,'2025-09-13 23:55:30');
INSERT INTO ``
('id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES
(5,'Ekaterina','ekaterina.petrova@outlook.com','F','2000-02-
11','145789',1.123,'2025-09-23 00:05:45');
INSERT INTO ``
('id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES
(6,'Paul','paul@superpochta.ru','M','1998-08-12','123789',1,'2025-09-23 00:05:45');

```

Задание 8

```

CREATE TABLE `simplifiedb`.`resume` (
  `resumeid` INT NOT NULL AUTO_INCREMENT,
  `userid` INT NOT NULL,
  `title` VARCHAR(100) NOT NULL,
  `skills` TEXT NULL,
  `created` TIMESTAMP NULL,
  PRIMARY KEY (`resumeid`),
  UNIQUE INDEX `resumeid_UNIQUE` (`resumeid` ASC) VISIBLE,
  INDEX `userid_idx` (`userid` ASC) VISIBLE,
  CONSTRAINT `userid`
  FOREIGN KEY (`userid`)
  REFERENCES `simplifiedb`.`users` (`id`)
  ON DELETE CASCADE
  ON UPDATE CASCADE);

```

При удалении записи из таблицы users СУБД автоматически каскадно удалит все связанные записи из таблицы resume для этого пользователя. Аналогично при обновлении значения id в таблицы users это изменение каскадно распространится на поле userid в таблице resume.

Задание 9

У одного пользователя может быть от 0 до неограниченного количества резюме (N). Ограничения по максимуму нет, минимум - 0 резюме.

```

/*
-- Query: SELECT * FROM simpledb.resume
LIMIT 0, 1000

-- Date: 2025-09-17 00:29
*/
INSERT INTO `` (`resumeid`,`userid`,`title`,`skills`,`created`) VALUES
(1,1,'Frontend Developer','JavaScript, React, HTML, CSS',NULL);
INSERT INTO `` (`resumeid`,`userid`,`title`,`skills`,`created`) VALUES
(2,1,'Fullstack Developer','Node.js, Express, MongoDB, React',NULL);
INSERT INTO `` (`resumeid`,`userid`,`title`,`skills`,`created`) VALUES
(3,2,'Backend Developer','Python, Django, PostgreSQL, Docker',NULL);

```

Операция завершится ошибкой. СУБД не позволит добавить запись с `userid=55`, поскольку нарушается целостность внешнего ключа - пользователя с таким `id` не существует в таблице `users`.

Operation failed: There was an error while applying the SQL script to the database.

Executing:

```

INSERT INTO `simpledb`.`resume` (`userid`, `title`, `skills`) VALUES ('55',
'gug', 'yufyu');

```

ERROR 1452: 1452: Cannot add or update a child row: a foreign key constraint fails (`simpledb`.`resume`, CONSTRAINT `userid` FOREIGN KEY (`userid`) REFERENCES `users` (`id`) ON DELETE CASCADE ON UPDATE CASCADE)

SQL Statement:

```

INSERT INTO `simpledb`.`resume` (`userid`, `title`, `skills`) VALUES ('55',
'gug', 'yufyu')

```

Задание 10

```

DELETE FROM `simpledb`.`resume` WHERE (`resumeid` = '3');

```

При удалении записей из таблицы `resume` не происходит никаких автоматических изменений в таблице `users`. Пользователи остаются в системе, но теряют свои резюме.

При изменении `id` пользователя в таблице `users` все соответствующие значения `userid` в таблице `resume` будут автоматически обновлены

id ^	na... ^	email	gender	bday	postal_code	rating	created	
3	Dima	Kij_Dima2003@mail.ru	NULL	NULL	NULL	NULL	0000-00-00 00:00:00	
4	Alex	Alex_Gj2007@mail.ru	M	NULL	575678	1.21	2025-09-13 23:55:30	
5	Ekaterina	ekaterina.petrova@outl...	F	2000-02-11	145789	1.123	2025-09-23 00:05:45	
6	Paul	paul@superpochta.ru	M	1998-08-12	123789	1	2025-09-23 00:05:45	
8	Katya	Huk_Kat1983@mail.ru	NULL	NULL	NULL	NULL	0000-00-00 00:00:00	
10	Lena	Bai_len2006@mail.ru	NULL	NULL	NULL	NULL	0000-00-00 00:00:00	
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	